Research Article

Petr Babkin and Oleg Bakhteev

Structure Aware Neural Architecture Search for Mixture of Experts

https://doi.org/10.1515/sample-YYYY-XXXX
Received Month DD, YYYY; revised Month DD, YYYY; accepted Month DD, YYYY

Abstract: The Mixture-of-Experts (MoE) layer, a sparsely activated neural architecture controlled by a routing mechanism, has recently achieved remarkable success across large-scale deep learning tasks. In parallel, Neural Architecture Search (NAS) has emerged as a powerful methodology for automatically discovering high-performing neural network. However, the application of NAS methods to MoE architectures remains an underexplored research area. In this work, we propose an architecture search framework for MoE models, which explicitly leverages the underlying cluster structure of the data. We evaluate the proposed approach on computer vision benchmarks and demonstrate that it outperforms baseline MoE architectures trained on the same datasets in terms of accuracy and computational efficiency.

Keywords: Mixture of Experts, Neural Architecture Search

1 Introduction

The Mixture-of-Experts (MoE) architecture, in which input is partitioned through a gating function and subsequently routed as weighted input to specialized experts [1–3], has been well-established in the literature for decades [4]. Scenarios in which the experts are classical statistical models are well characterized in the literature [5], particularly when the experts are modeled as logistic regressors [6] or as Gaussian models [7, 8]. In recent years, concurrent with the growing interest in neural networks, researchers began exploring the application of MoE structures to deep learning architectures [9]. This efficiency was achieved through sparse gating mechanisms, where the input is routed only to a fixed number of experts, which is significantly smaller than the complete set [10–12]. MoE architectures found widespread adoption in the transformer domain, where model size plays a crucial role in achieving superior performance [13–15].

Simultaneously, the field of Neural Architecture Search (NAS) has developed into a well-studied area [16–18]. NAS aims to select optimal architectures from a wide range of candidates, addressing the challenge that architecture selection typically requires substantial domain expertise and manual tuning [16, 19]. The field has diverged into two main paradigms. The first is computationally intensive many-shot NAS, which involves multiple training runs followed by candidate selection using methods such as reinforcement learning [20, 21] and evolutionary algorithms [22, 23]. The second is one-shot NAS, where a single supernet is trained, and the optimal subnet is subsequently selected [19, 24, 25]. This approach effectively transforms the search problem into finding optimal compression strategies for the supernet [19].

Despite the individual successes of both MoE and NAS methodologies, the application of NAS techniques specifically to MoE architectures remains a relatively underexplored research area, presenting significant opportunities for advancing both efficiency and performance in large-scale neural architectures. In contrast to the classical approach where all experts share the same architecture [12], our study allows architectural heterogeneity between experts. We claim that this flexibility enhances the overall efficiency of the resulting architecture by enabling more specialized and adaptable expert networks.

Petr Babkin, Oleg Bakhteev, Moscow Institute of Physics and Technology, Intelligent Systems, Moscow, Russia, e-mail: babkin.pk@phystech.edu

It is important to note that several studies in the literature have addressed the problem of discovering or designing architectures for mixture-of-experts models. In the article [26], the authors employed a comprehensive combination of approaches to discover a fast-operating Mixture-of-Experts (MoE) architecture. The expert weights are derived from a supernet, and the search process is evolutionary in nature. There is a practical work [27] in which the authors develop a framework for efficient MoE inference tailored to specific hardware. The work most closely related to ours is [28], where the authors conduct a search by varying the number of experts in the layers. In contrast to this study, we propose to allow greater flexibility by optimizing the model architectures themselves rather than only the number of experts.

The MoE architecture is inherently well-suited for data with strong clustering properties. There exist theoretical studies, which demonstrate on relatively simple examples that this claim holds [29]. Additionally, practical researches show the effectiveness of MoE on multimodal datasets [30–32]. However, despite these promising results, the underlying mechanisms that enable experts to align with distinct modalities remain insufficiently understood, motivating further investigation into the principles governing modality-specific specialization within MoE frameworks.

Given the foregoing, we formulate the following research question: *how can we design conditions under which expert architectures adapt to specific data clusters?* In this study, we aim to address this question by examining how models evolve during adaptation to specific data modalities.

Contributions

- **SA-NAS framework:** We propose a Structure-Aware Neural Architecture Search (SA-NAS) framework that optimizes expert architectures with respect to the clustering structure of the data.
- **Theoretical understanding:** We provide a theoretical foundation for our method, demonstrating how the data's clustering structure influences the final architectures identified by the search algorithm.

2 Related Work

Mixture-of-Experts. The Mixture-of-Experts (MoE) framework was originally introduced in [1–3] as a way to combine specialised statistical models through a learnable gating mechanism. Subsequent theoretical analyses characterised identifiability and convergence in classical MoE formulations [5–8]. With the rise of deep learning, MoE has been revived in the form of sparsely activated neural networks, where the gate routes each input to only a small subset of experts [9–12]. This sparse formulation enables training of models with hundreds of billions of parameters while keeping per-token compute roughly constant, and is now a standard building block of modern large-scale transformers [13–15].

Neural Architecture Search. Neural Architecture Search (NAS) automates the design of neural networks and has matured into a wide research field [16]. Many-shot approaches based on reinforcement learning [18, 20, 21] or evolutionary search [22, 23] explicitly train candidate architectures and select the best performer. One-shot methods reduce the search cost by training a single supernet that contains all candidate architectures as sub-networks [17, 19, 24, 25]. DARTS [19] is the most prominent representative of this family and forms one of the baselines we compare against in our experimental study.

Neural Architecture Search for Mixture-of-Experts. The intersection of NAS and MoE is a comparatively recent and still sparsely populated research area. AutoMoE [26] applies an evolutionary search on top of a shared supernet to obtain heterogeneous MoE architectures for neural machine translation. CMN [27] co-designs MoE topologies for computing-in-memory hardware. The work most closely related to ours is MoE-NAS [28], which optimises the number of experts per layer but keeps the expert architectures shared. In contrast, we explicitly search over per-expert architectures and tie this search to the cluster structure of the input data.

Theoretical Understanding. A growing body of theoretical work studies why MoE models perform well on data with strong cluster or modality structure. [29] proves on simplified settings that MoE provably benefits from clusterable data, while [33] analyses the optimisation landscape of sparse gates. Empirical evidence from multimodal and multi-task learning [30–32, 34] further supports the claim that experts naturally specialise to distinct sub-distributions. Our work builds on this perspective and turns it into an explicit search objective by optimising over *per-cluster* expert architectures rather than treating the cluster structure as a by-product of training.

3 Problem Statement

3.1 Neural Architecture Search

Definition 1. *Let $\mathcal{V} = \{1, 2, \dots, N\}$ be a set of N vertices representing nodes in a directed acyclic graph (DAG). We define a set of edges as*

$$\mathcal{E} = \{(i, j) \in \mathcal{V} \times \mathcal{V} \mid i < j\}, \tag{1}$$

representing the possible connections between vertices, where the ordering constraint $i < j$ enforces a topological structure.

Definition 2. *The set of admissible operations $\mathcal{O}$ comprises a collection of differentiable or discrete operations that can be applied to edges in the network architecture. Common operations include convolutions of varying kernel sizes (e.g., 1×1, 3×3), pooling operations (max pooling, average pooling), and skip connections.*

For each edge $(i, j) \in \mathcal{E}$, an operation $o^{(i,j)} \in \mathcal{O}$ is assigned to transmit information from vertex i to vertex j. The task of neural architecture search is thus reduced to determining the optimal assignment of operations to edges.

Definition 3. *Let $\boldsymbol{\alpha} \in \mathcal{A}$ be an architecture vector encoding the operations assigned to each edge, where $\mathcal{A}$ is the search space comprising all possible architecture configurations. Each component $\alpha_{i,j}$ specifies the operation on edge (i, j).*

For a given architecture $\boldsymbol{\alpha}_k \in \mathcal{A}$ and its corresponding parameters $\boldsymbol{w}_{\boldsymbol{\alpha}_k} \in \mathcal{W}_{\boldsymbol{\alpha}_k}$, we denote the neural network as $f(\boldsymbol{\alpha}_k, \boldsymbol{w}_{\boldsymbol{\alpha}_k}) : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}$ and $\mathcal{Y}$ are the input and output spaces respectively.

Definition 4. *The empirical loss functions on training and validation datasets are defined as follows:*

$$\mathcal{L}_{train}(f) = \sum_{(\boldsymbol{x}, y) \in \mathcal{D}_{train}} \ell(f(\boldsymbol{x}), y), \quad \mathcal{L}_{val}(f) = \sum_{(\boldsymbol{x}, y) \in \mathcal{D}_{val}} \ell(f(\boldsymbol{x}), y), \tag{2}$$

where $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$ is a task-specific loss function, and $\mathcal{D}_{train}$ and $\mathcal{D}_{val}$ denote the training and validation datasets respectively. We use the shorthand notation $\mathcal{L}_{train}(\boldsymbol{\alpha}_k, \boldsymbol{w}_{\boldsymbol{\alpha}_k})$ and $\mathcal{L}_{val}(\boldsymbol{\alpha}_k, \boldsymbol{w}_{\boldsymbol{\alpha}_k})$ interchangeably with $\mathcal{L}_{train}(f(\boldsymbol{\alpha}_k, \boldsymbol{w}_{\boldsymbol{\alpha}_k}))$ and $\mathcal{L}_{val}(f(\boldsymbol{\alpha}_k, \boldsymbol{w}_{\boldsymbol{\alpha}_k}))$ respectively.

The neural architecture search problem is formulated as a bilevel optimization problem:

$$\begin{aligned} &\min_{\boldsymbol{\alpha} \in \mathcal{A}} \mathcal{L}_{\text{val}}(\boldsymbol{\alpha}, \boldsymbol{w}^*_{\boldsymbol{\alpha}}), \\ \text{s.t.} \quad &\boldsymbol{w}^*_{\boldsymbol{\alpha}} = \arg \min_{\boldsymbol{w} \in \mathcal{W}_{\boldsymbol{\alpha}}} \mathcal{L}_{\text{train}}(\boldsymbol{\alpha}, \boldsymbol{w}), \end{aligned} \tag{3}$$

where $\mathcal{W}_{\boldsymbol{\alpha}}$ denotes the space of all learnable parameters for architecture $\boldsymbol{\alpha}$.

Interpretation: The outer minimization seeks an architecture $\boldsymbol{\alpha}$ that minimizes validation loss. The constraint ensures that the network weights $\boldsymbol{w}^*_{\boldsymbol{\alpha}}$ are optimally trained on the training dataset for the selected architecture. This formulation reflects the fundamental trade-off between model expressiveness (architecture choice) and generalization capability (validation performance).

3.2 Mixture-of-Experts Architecture

Definition 5. *A Mixture-of-Experts (MoE) model is a composite model comprising K expert networks, each specialized for a distinct subset of the input space. Given a dataset partitioned into K clusters, $\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2 \cup \cdots \cup \mathcal{D}_K$ (forming a partition), where each cluster $\mathcal{D}_k$ corresponds to a specialized domain or data modality.*

Let $\boldsymbol{\alpha} = [\boldsymbol{\alpha}_1^T, \boldsymbol{\alpha}_2^T, \ldots, \boldsymbol{\alpha}_K^T]^T$ be the concatenation of architecture vectors for all K experts. We denote a Mixture-of-Experts model with architecture $\boldsymbol{\alpha}$ and parameters $\boldsymbol{w}_{\boldsymbol{\alpha}}$ as $g(\boldsymbol{\alpha}, \boldsymbol{w}_{\boldsymbol{\alpha}})$.

The MoE prediction is computed as a weighted combination of expert outputs:

$$g(\boldsymbol{\alpha}, \boldsymbol{w}_{\boldsymbol{\alpha}}, \boldsymbol{w}_r)(\boldsymbol{x}) = \sum_{k=1}^{K} r_k(\boldsymbol{x}, \boldsymbol{w}_r) \cdot f(\boldsymbol{\alpha}_k, \boldsymbol{w}^*_{\boldsymbol{\alpha}_k})(\boldsymbol{x}), \tag{4}$$

where $f(\boldsymbol{\alpha}_k, \boldsymbol{w}^*_{\boldsymbol{\alpha}_k})$ denotes the k-th expert network, and $r_k(\boldsymbol{x}, \boldsymbol{w}_r) \geq 0$ are routing gate values learned by a gating network with parameters $\boldsymbol{w}_r$. The routing gates typically satisfy $\sum_{k=1}^{K} r_k(\boldsymbol{x}, \boldsymbol{w}_r) = 1$.

3.3 Neural Architecture Search for Mixture-of-Experts

The problem of discovering optimal architectures for each expert in a Mixture-of-Experts model can be formulated as an extension of Problem 3 to the multi-expert setting.

$$\begin{aligned} &\min_{\boldsymbol{\alpha} \in \mathcal{A}^K} \mathcal{L}_{\text{val}}(\boldsymbol{\alpha}, \boldsymbol{w}^*_{\boldsymbol{\alpha}}, \boldsymbol{w}_r), \\ \text{s.t.} \quad &(\boldsymbol{w}^*_{\boldsymbol{\alpha}}, \boldsymbol{w}^*_r) = \arg \min_{\boldsymbol{w}, \boldsymbol{w}_r \in \mathcal{W}_{\boldsymbol{\alpha}} \times \mathcal{W}_r} \mathcal{L}_{\text{train}}(\boldsymbol{\alpha}, \boldsymbol{w}, \boldsymbol{w}_r) \end{aligned} \tag{5}$$

where $\mathcal{A}^K$ denotes the combined search space for K expert architectures.

Remark 1. *Problem 5 extends the search space from $\mathcal{A}$ to $\mathcal{A}^K$, exponentially increasing computational complexity. To mitigate this complexity, we propose the following decomposition.*

We reformulate Problem 5 by decomposing the global optimization objective into cluster-wise sub-problems:

$$\begin{aligned} &\min_{\boldsymbol{\alpha} \in \mathcal{A}^K} \sum_{k=1}^{K} \mathcal{L}^{\mathcal{D}_k}_{\text{val}}(\boldsymbol{\alpha}_k, \boldsymbol{w}^*_{\boldsymbol{\alpha}_k}), \\ \text{s.t.} \quad &\forall k \in \{1, 2, \ldots, K\}, \quad \boldsymbol{w}^*_{\boldsymbol{\alpha}_k} = \arg \min_{\boldsymbol{w} \in \mathcal{W}_{\boldsymbol{\alpha}_k}} \mathcal{L}^{\mathcal{D}_k}_{\text{train}}(\boldsymbol{w}, \boldsymbol{\alpha}_k), \end{aligned} \tag{6}$$

where $\mathcal{L}^{\mathcal{D}_k}_{\text{val}}$ and $\mathcal{L}^{\mathcal{D}_k}_{\text{train}}$ denote validation and training losses computed on cluster $\mathcal{D}_k$ respectively.

Motivation: This decomposition assumes that the optimal expert for cluster $\mathcal{D}_k$ can be found independently by optimizing the architecture and weights on the k-th cluster's data. This assumption is justified when clusters represent sufficiently distinct data distributions and the experts operate independently (no parameter sharing across experts beyond the gating network).

3.4 Cluster-aware MoE Objective

The decomposed Problem 6 still requires training one architecture per cluster to evaluate every candidate. To turn architecture search into a tractable optimisation problem we introduce a function $u : \mathcal{A} \times [0,1]^M \to \mathbb{R}_{\geq 0}$ that scores the validation accuracy of an architecture $\boldsymbol{\alpha}$ trained on a subset of clusters, encoded by a vector $\mathbf{R} \in [0,1]^M$. A concrete construction of u as a learned surrogate is given in Section 4.1.

Let $r_{mk} \in [0,1]$ be the soft assignment probability of cluster m to expert k, with $\sum_{k=1}^{K} r_{mk} = 1$, and collect them in a routing matrix $\mathbf{r} \in [0,1]^{M \times K}$. Denote by $\mathbf{R}_k = \mathbf{r}_{:,k} \in [0,1]^M$ the column corresponding to

expert k. The cluster-aware MoE optimisation problem is

$$\mathcal{J}(\boldsymbol{\alpha}_{1:K}, \mathbf{r}) \;=\; \sum_{m=1}^{M} \log \sum_{k=1}^{K} r_{mk}\, u(\boldsymbol{\alpha}_k, \mathbf{R}_k) \;\to\; \max_{\boldsymbol{\alpha}_{1:K},\, \mathbf{r}} . \tag{7}$$

Intuitively, r_{mk} specifies how strongly cluster m is routed to expert k, while u scores how well each expert handles the cluster subset that it is responsible for. Problem (7) is the empirical surrogate-based counterpart of the marginal MoE log-likelihood $\sum_{n=1}^{N} \log \sum_{k=1}^{K} r_{nk}\, p(y_n, n \in C_k \mid x_n, \boldsymbol{\alpha}_k, \boldsymbol{\theta}_k)$, where the per-cluster likelihood factor has been replaced by the quality estimate $u(\boldsymbol{\alpha}_k, \mathbf{R}_k)$. The algorithm of Section 4.2 solves (7) via Generalised EM with an adaptively refined surrogate.

4 Main Part

4.1 Surrogate Function

Direct optimisation of Problem 6 requires repeatedly training neural networks for many architecture candidates, which is prohibitively expensive in the MoE setting where each evaluation already involves K networks. To address this computational bottleneck we replace exact training by a learned surrogate model.

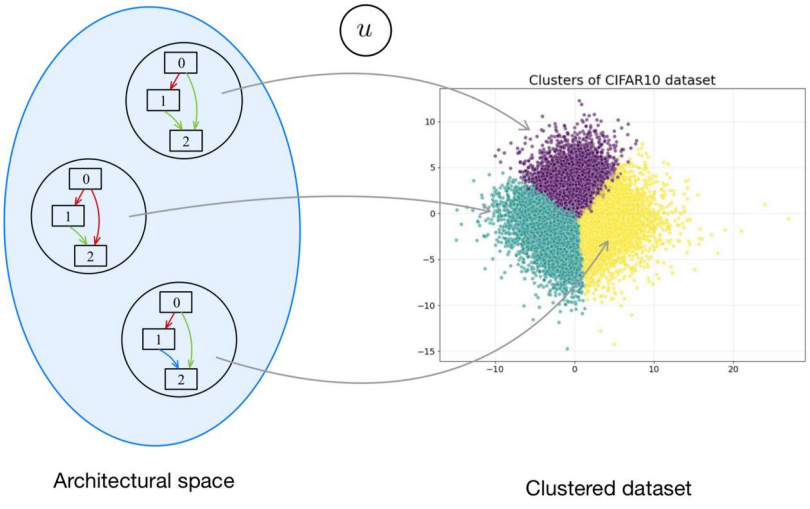

Fig. 1: Method workflow: a surrogate model u is used to select the most suitable expert architectures for each cluster.

Definition 6. *Let the training data be partitioned into M clusters $\mathcal{D}_{train} = \mathcal{C}_1 \cup \cdots \cup \mathcal{C}_M$, and for $\boldsymbol{b} \in \{0,1\}^M$ let $\mathcal{D}_{\boldsymbol{b}} = \bigcup_{m:b_m=1} \mathcal{C}_m$ be the cluster subset selected by $\boldsymbol{b}$. A* surrogate function

$$u : \mathcal{A} \times \{0,1\}^M \to \mathbb{R}, \qquad u(\boldsymbol{\alpha}, \boldsymbol{b}) \approx \mathrm{acc}_{\mathrm{val}}\big(f(\boldsymbol{\alpha}, \boldsymbol{w}^*_{\boldsymbol{\alpha},\boldsymbol{b}})\big)\,, \tag{8}$$

predicts the validation accuracy of architecture $\boldsymbol{\alpha}$ when trained on $\mathcal{D}_{\boldsymbol{b}}$, without performing the actual training.

The surrogate is trained on a dataset of architecture/subset pairs collected by genuine training:

$$\mathcal{D}_{\mathrm{surr}} = \{(\boldsymbol{\alpha}^{(i)}, \boldsymbol{b}^{(i)}, y^{(i)})\}_{i=1}^{N}, \qquad y^{(i)} = \mathrm{acc}_{\mathrm{val}}\big(f(\boldsymbol{\alpha}^{(i)}, \boldsymbol{w}^*_{\boldsymbol{\alpha}^{(i)},\boldsymbol{b}^{(i)}})\big). \tag{9}$$

We parametrise $u_{\boldsymbol{\theta}}$ as a Graph Neural Network encoder over the architecture DAG concatenated with an embedding of the cluster mask $\boldsymbol{b}$, and minimise the mean squared regression loss

$$\min_{\boldsymbol{\theta}} \frac{1}{|\mathcal{D}_{\text{surr}}|} \sum_{(\boldsymbol{\alpha},\boldsymbol{b},y)\in\mathcal{D}_{\text{surr}}} \big(u_{\boldsymbol{\theta}}(\boldsymbol{\alpha},\boldsymbol{b}) - y\big)^2. \tag{10}$$

The surrogate is refined inside an active learning loop. At every iteration we use Monte-Carlo dropout to obtain a predictive mean $\mu(\boldsymbol{\alpha},\boldsymbol{b})$ and standard deviation $\sigma(\boldsymbol{\alpha},\boldsymbol{b})$ over T stochastic forward passes, and score candidates by the Upper Confidence Bound $\text{UCB} = \mu + \sigma$. The most promising candidate is trained from scratch, the resulting triple is appended to $\mathcal{D}_{\text{surr}}$, and the surrogate is refit. The crucial property of $u_{\boldsymbol{\theta}}$ is that it is trained *only on discrete* masks $\boldsymbol{b}$; differentiable optimisation must therefore be combined with a relaxation of the discrete inputs.

4.2 Surrogate-assisted Generalized EM (SAGEM)

Treating the cluster→expert routing as a latent variable yields a natural Expectation–Maximisation procedure on (7), in which the surrogate $u^{(t)}$ replaces the otherwise-intractable expert quality.

E-step. For fixed $(\boldsymbol{\alpha}_{1:K}, \mathbf{r})$, compute responsibilities

$$q_{mk} \;\propto\; r_{mk}\, u(\boldsymbol{\alpha}_k, \mathbf{R}_k), \qquad \sum_{k=1}^{K} q_{mk} = 1. \tag{11}$$

M-step (routing). Maximise $\sum_{m,k} q_{mk}\big[\log r_{mk} + \log u(\boldsymbol{\alpha}_k, \mathbf{R}_k)\big]$ over $\mathbf{r}$ via gradient ascent on the logits, again with a straight-through Gumbel-Softmax for $\mathbf{R}_k$.

M-step (architectures). For each expert k sample new candidates $\boldsymbol{\alpha}_k^{\text{new}}$ from the search space and accept whenever $u(\boldsymbol{\alpha}_k^{\text{new}}, \mathbf{R}_k) > u(\boldsymbol{\alpha}_k, \mathbf{R}_k)$.

Because the surrogate is biased towards the strongest expert seen during data collection, the EM dynamics tend to collapse all clusters onto a single expert. We mitigate this by adding to the M-step objective a load-balancing penalty

$$\text{LB}(\mathbf{r}) \;=\; K \cdot \sum_{k=1}^{K} \bar{r}_k \cdot p_k, \qquad \bar{r}_k = \frac{1}{M}\sum_{m=1}^{M} r_{mk}, \quad p_k = \frac{|\{m : \arg\max_{k'} r_{mk'} = k\}|}{M}, \tag{12}$$

weighted by a coefficient λ_{LB}. This penalty is minimal when the routing is uniform across experts and saturates at K under full collapse, providing the regulariser used in our experiments.

Algorithm 1 Surrogate-assisted Generalized EM (SAGEM) for MoE Architecture Search

Require: Surrogate $u_{\boldsymbol{\theta}}$, search space $\mathcal{A}$, number of experts K, clusters M, weight λ_{LB}, EM iterations T.

1: Initialise architectures $\boldsymbol{\alpha}_1, \dots, \boldsymbol{\alpha}_K$ at random and logits $\boldsymbol{\ell} \in \mathbb{R}^{M\times K} \sim \mathcal{N}(\mathbf{0}, I)$.
2: **for** $t = 1$ to T **do**
3: **E-step.** $q_{mk} \propto r_{mk}\, u(\boldsymbol{\alpha}_k, \mathbf{R}_k)$.
4: **M-step (r).** Gradient ascent on $\sum q_{mk}[\log r_{mk} + \log u] - \lambda_{\text{LB}}\,\text{LB}(\mathbf{r})$ using straight-through Gumbel-Softmax for $\mathbf{R}_k$.
5: **M-step ($\boldsymbol{\alpha}$).** For each expert k, sample candidates and keep the best under $u(\cdot, \mathbf{R}_k)$.
6: **Active learning step.** Train selected candidates, update $\mathcal{D}_{\text{surr}}$, refit $u_{\boldsymbol{\theta}}$.
7: **end for**
8: **return** $\boldsymbol{\alpha}_{1:K}$ and hard assignment $\hat{r}_{mk} = 1, [k = \arg\max_{k'} r_{mk'}]$.

4.3 Convergence Analysis

A natural concern with Algorithm 1 is that the surrogate $u^{(t)}$ used inside the E- and M-steps is itself updated across iterations, so the classical monotonicity argument for EM [35, 36] no longer applies directly. We show that, provided the surrogate error decays fast enough, SAGEM still converges in the same sense as Generalised EM.

Let u_{true} denote the (unknown) ground-truth expert quality. Define the per-iteration surrogate error

$$\varepsilon_t \;=\; \sup_{\boldsymbol{\alpha}\in\mathcal{A},\; \boldsymbol{R}\in[0,1]^M} \bigl|\log u^{(t)}(\boldsymbol{\alpha},\boldsymbol{R}) \;-\; \log u_{\text{true}}(\boldsymbol{\alpha},\boldsymbol{R})\bigr|, \tag{13}$$

the parameter $\theta = (\boldsymbol{\alpha}_{1:K}, \mathbf{r})$ on the compact set $\Theta = \mathcal{A}^K \times (\Delta_K)^M$, the true log-likelihood $L_{\text{true}}(\theta) = \sum_m \log \sum_k r_{mk} u_{\text{true}}(\boldsymbol{\alpha}_k, \mathbf{R}_k)$ and the *true* Q-function $Q_{\text{true}}(\theta;\theta') = \sum_{m,k} w_{mk}(\theta') \log\bigl(r_{mk}\, u_{\text{true}}(\boldsymbol{\alpha}_k, \mathbf{R}_k)\bigr)$ with $w_{mk}(\theta')$ the responsibilities computed under u_{true}.

Assumption 1. *There exist $0 < c \le C < \infty$ such that $c \le u_{\text{true}}(\boldsymbol{\alpha},\mathbf{R})$, $u^{(t)}(\boldsymbol{\alpha},\mathbf{R}) \le C$ for all $\boldsymbol{\alpha}\in\mathcal{A}$, $\mathbf{R}\in[0,1]^M$, $t \ge 0$.*

Assumption 2. *The surrogate errors are summable: $\sum_{t=0}^{\infty} \varepsilon_t < \infty$.*

Assumption 3. *The M-step of Algorithm 1 is generalised, i.e. for every t it returns $\theta^{(t+1)}$ with $\mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t+1)}) \ge \mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t)})$, where $\mathcal{F}_v(q,\theta) = \sum_{m,k} q_{mk} \log\bigl(r_{mk} v(\boldsymbol{\alpha}_k, \mathbf{R}_k)/q_{mk}\bigr)$ is the variational free energy.*

Assumption 4. *For every $\boldsymbol{\alpha}\in\mathcal{A}$ the map $\mathbf{R} \mapsto u_{\text{true}}(\boldsymbol{\alpha},\mathbf{R})$ is continuous on $[0,1]^M$.*

Theorem 1 (Convergence of SAGEM with adaptive surrogate). *Under Assumptions 1–4, the iterates $\{\theta^{(t)}\}$ produced by Algorithm 1 satisfy*

(a) *the sequence $\{L_{\text{true}}(\theta^{(t)})\}$ converges;*

(b) *every limit point $\theta^\star$ is a stationary point of L_{true} on Θ, in the sense $Q_{\text{true}}(\theta;\theta^\star) \le Q_{\text{true}}(\theta^\star;\theta^\star)$ for every $\theta\in\Theta$.*

The proof, together with the auxiliary lemmas (uniform deviation of the ELBO, stability of the softmax responsibilities, and a quasi-monotonicity bound $L_{\text{true}}(\theta^{(t+1)}) \ge L_{\text{true}}(\theta^{(t)}) - 2M\varepsilon_t$ established via a Robbins–Siegmund argument), is given in Appendix A.

Remark 2. *The summability assumption 2 is the surrogate analogue of the diminishing step-size condition in stochastic EM [35]; replacing it by the weaker condition $\varepsilon_t \to 0$ preserves part (b) of Theorem 1 but not part (a). Practically, summability is achieved whenever the active learning step in Algorithm 1 provides sufficiently rich coverage of $\mathcal{A}\times[0,1]^M$.*

5 Experiments

We evaluate the proposed framework on two complementary problems: (i) a controlled 2D toy task that allows us to verify whether the recovered routing matches the ground-truth cluster structure, and (ii) the CIFAR-100 image classification benchmark, where we compare against several MoE and NAS baselines.

5.1 Toy Experiment

We sample 2000 points in $\mathbb{R}^2$ from a four-class problem consisting of one "linear" cloud (two classes separated by a diagonal) and one "ring" cloud (two classes corresponding to an inner and an outer ring). The data is split 80/20 into training/validation and clustered into $M = 20$ KMeans clusters, yielding the partition described in our data preparation pipeline. We use $K = 2$ experts so that the ideal solution is unambiguous: one expert should specialise to the linear cloud and the other to the ring cloud. The search space is the cell-based DARTS-style space of [19], restricted to small operations suitable for 2D inputs.

With the parameters reported in our experimental log, SAGEM (Algorithm 1) recovers the *ideal* cluster split $(10, 10)$ between the two experts in 20 EM iterations, matching the hand-defined oracle assignment that splits clusters according to the centroid coordinate $x_0 = -2$. This sanity check confirms that the surrogate-guided objective in (7) is well posed and that the proposed optimiser solves it when the surrogate is sufficiently accurate, in agreement with the convergence guarantee of Theorem 1.

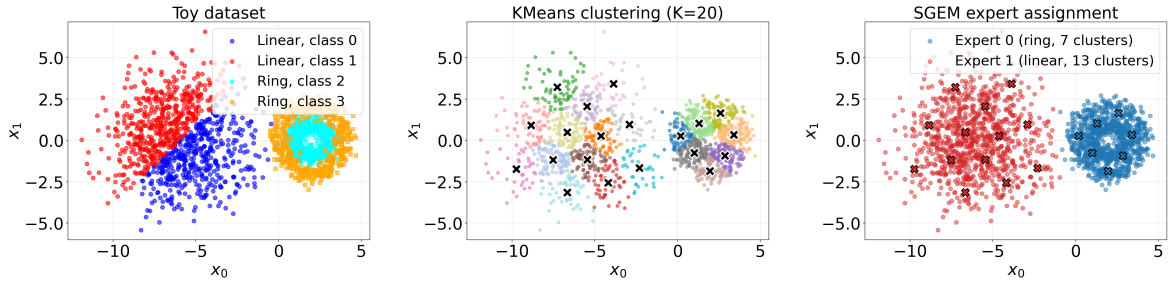

Fig. 2: Toy experiment. (a) The four-class dataset: a linear cloud (classes 0–1) and a ring cloud (classes 2–3). (b) The $M = 20$ KMeans clusters (crosses mark the centroids). (c) The expert assignment recovered by SAGEM with $K = 2$: it splits the clusters exactly along the linear/ring boundary, matching the oracle partition up to a permutation of expert labels.

5.2 CIFAR-100 and SVHN mixture

To test whether the cluster-aware objective recovers a meaningful data partition on realistic data, we build a heterogeneous benchmark by mixing CIFAR-100 and SVHN in equal proportions (42 000 images total). SVHN labels are shifted by 100, so the combined problem has 110 classes. The two sources form a maximally separated domain structure, which gives a well-defined oracle: with $K = 2$ experts the ideal routing assigns each cluster to the expert specialised on its source. We partition the data into $M = 30$ *semantic* clusters obtained from a pretrained feature extractor, using a three-way train/validation/test split (0.7/0.15/0.15). KMeans is fit on the training embeddings only; validation and test points are assigned to the nearest centroid. The resulting oracle partition is 19 CIFAR-majority and 11 SVHN-majority clusters, with a mean source purity of 0.978.

All MoE configurations use $K = 2$ experts with `init_channels = 16`. The base network is a small DARTS-style architecture `CIFAR100Net` (stem, two normal cells, one fixed reduction cell, global average pooling, fully connected head). The MoE wrapper instantiates K such networks combined through a fixed hard *cluster gate*: each cluster is routed to a single expert, so the gate directly reflects the discovered data partition. Architecture search uses the train/validation split; the final model is retrained for 100 epochs on train $\cup$ validation and evaluated *once* on the held-out test set. We report test accuracy averaged over five seeds. Training is performed inside a Docker container with PyTorch 2.5.1 on a single RTX 3080 Ti GPU.

We compare against the following methods, all evaluated with the same cluster gate:

- **Random-MoE**: 200 random architecture pairs are searched at 30 epochs each on a random cluster split, then the best is retrained.
- **DARTS**$\times K$: K experts that share the same DARTS-discovered architecture on a random cluster split.
- **SAGEM**: our method with the Phase C uniform-mixture coefficient $p_C = 0.5$ and load-balancing weight $\lambda_{\text{LB}} = 50$.

Table 1 reports the test accuracy with cluster gating. The differences between the methods are small and lie within one standard deviation of each other: SAGEM (0.5935 ± 0.003) is statistically indistinguishable from DARTS$\times K$ (0.5913 ± 0.009), and both improve only marginally over the random-MoE baseline

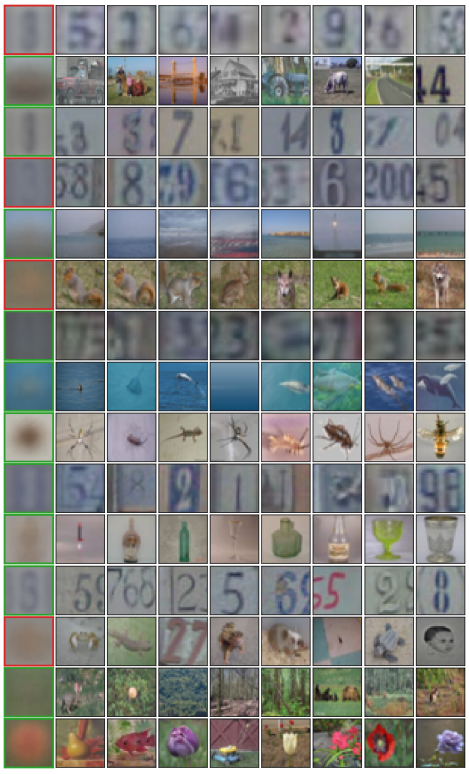

Fig. 3: Per-cluster representatives of the CIFAR-100 / SVHN mixture under the SAGEM routing ($K = 2$, seed 322; first 15 of the $M = 30$ semantic clusters shown). Each row is one cluster: the leftmost column is the cluster medoid and the rest are the eight images nearest to the cluster mean. The medoid border (leftmost column) is coloured by the expert assigned by SAGEM (E0 red, E1 green).

(0.5551 ± 0.031, with a large seed-to-seed variance). Hard cluster gating partitions the data so that each expert sees fewer examples, which largely offsets the benefit of specialisation, leaving the learned routing of SAGEM no better than sharing a single architecture across experts on a random split.

Tab. 1: Test accuracy on the CIFAR-100 / SVHN mixture with cluster gating ($M = 30$, $K = 2$, semantic clustering, mean $\pm$ std over five seeds).

Method	test acc. (cluster gate)
Random-MoE	0.5551 ± 0.031
DARTS$\times K$ (same arch)	0.5913 ± 0.009
SAGEM, λ_{LB}=50, p_C=0.5	$\mathbf{0.5935 \pm 0.003}$

Because the oracle partition by data source is sharp (mean purity 0.978), we can directly check whether the learned routing aligns with it (Figure 3). Across five seeds, the SAGEM hard assignment agrees with the oracle source split on only $\approx 60\%$ of clusters (best matching of the two label permutations), barely above the $\approx 54\%$ baseline of a balanced random split. The load-balancing penalty is the main culprit: the oracle split is imbalanced (19/11) whereas the penalty pulls the routing towards 15/15, washing out the source signal. Hence the hypothesis that the cluster-aware objective discovers the underlying data domains is *not* supported on this benchmark.

With cluster gating, all MoE variants cluster tightly around 0.59 test accuracy, and the source-recovery analysis shows that the discovered partition is essentially uncorrelated with the true data structure. We

attribute this to a self-reinforcing bias in the surrogate dataset rather than to the EM optimiser itself: the active-learning loop over-samples cluster masks favourable to a dominant expert, so the routing is driven by the surrogate's bias and is held balanced only by the load-balancing penalty. Removing this bias — for instance by symmetrising the data collection across experts or enforcing permutation invariance of the surrogate — is the most promising direction for making the cluster-aware routing informative. As Figure 3 illustrates at the level of individual images, many SVHN-dominated clusters (digit crops) and CIFAR-dominated clusters (natural images) are routed to different experts, but the assignment is far from a clean source split, consistent with the $\approx 60\%$ agreement reported above.

6 Conclusion

We presented SA-NAS, a Structure-Aware Neural Architecture Search framework for Mixture-of-Experts models, which couples per-expert architecture search with the cluster structure of the underlying data. The framework relies on a Graph Neural Network surrogate that predicts validation accuracy from an architecture description and a binary cluster mask, trained inside an active-learning loop with Monte-Carlo dropout uncertainty estimates. Three optimisation algorithms (sampling, Gumbel-Softmax gradient and EM) instantiate the same cluster-aware MoE objective and share a common implementation of the surrogate.

On the controlled toy problem the EM variant recovers the ideal linear/ring split with $K = 2$ experts, confirming that the formulation is well posed when the surrogate is reliable. On CIFAR-100 with semantic clustering, $K = 3$ experts and a sufficient load-balancing weight, SAGEM produces balanced routings and reaches 0.4758 validation accuracy — 4.7 p.p. above a single DARTS-discovered network and 1.6 p.p. above a learnable-gate random MoE in the best run, although averaged over three seeds the gain over random MoE is essentially zero. Our diagnostic analysis shows that this brittleness comes from a self-reinforcing bias in the surrogate dataset rather than from the EM optimiser itself.

The most promising direction for future work is therefore to remove this bias — for example by enforcing permutation invariance of the surrogate across experts, or by symmetrising the active-learning data collection so that every expert sees the same distribution of cluster masks. We also plan to scale the search space, to extend the framework to transformer-based MoE layers, and to develop tighter theoretical guarantees for the cluster-aware objective when the surrogate is consistent.

References

[1] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.

[2] Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural computation*, 6 (2):181–214, 1994.

[3] Robert A Jacobs, Fengchun Peng, and Martin A Tanner. A bayesian approach to model selection in hierarchical mixtures-of-experts architectures. *Neural Networks*, 10(2):231–241, 1997.

[4] Siyuan Mu and Sen Lin. A comprehensive survey of mixture-of-experts: Algorithms, theory, and applications. *arXiv preprint arXiv:2503.07137*, 2025.

[5] Jakub Piwko, Jędrzej Ruciński, Dawid Płudowski, Antoni Zajko, Patryzja Żak, Mateusz Zacharecki, Anna Kozak, and Katarzyna Woźnica. Divide, specialize, and route: A new approach to efficient ensemble learning. *arXiv preprint arXiv:2506.20814*, 2025.

[6] Huy Nguyen, Pedram Akbarian, TrungTin Nguyen, and Nhat Ho. A general theory for softmax gating multinomial logistic mixture of experts. *arXiv preprint arXiv:2310.14188*, 2023.

[7] Huy Nguyen, TrungTin Nguyen, and Nhat Ho. Demystifying softmax gating function in gaussian mixture of experts. *Advances in Neural Information Processing Systems*, 36:4624–4652, 2023.

[8] Carl Rasmussen and Zoubin Ghahramani. Infinite mixtures of gaussian process experts. *Advances in neural information processing systems*, 14, 2001.

[9] David Eigen, Marc'Aurelio Ranzato, and Ilya Sutskever. Learning factored representations in a deep mixture of experts. *arXiv preprint arXiv:1312.4314*, 2013.

[10] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[11] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. *Advances in Neural Information Processing Systems*, 34:8583–8595, 2021.

[12] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[13] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[14] Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models. *arXiv preprint arXiv:2501.11873*, 2025.

[15] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

[16] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojiang Chen, and Xin Wang. A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Computing Surveys (CSUR)*, 54(4):1–34, 2021.

[17] Niv Nayman, Asaf Noy, Tal Ridnik, Itamar Friedman, Rong Jin, and Lihi Zelnik. Xnas: Neural architecture search with expert advice. *Advances in neural information processing systems*, 32, 2019.

[18] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V Le. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8697–8710, 2018.

[19] Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018.

[20] Barret Zoph and Quoc V Le. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2016.

[21] Bowen Baker, Otkrist Gupta, Nikhil Naik, and Ramesh Raskar. Designing neural network architectures using reinforcement learning. *arXiv preprint arXiv:1611.02167*, 2016.

[22] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. Regularized evolution for image classifier architecture search. In *Proceedings of the aaai conference on artificial intelligence*, volume 33, pages 4780–4789, 2019.

[23] Zhaohui Yang, Yunhe Wang, Xinghao Chen, Boxin Shi, Chao Xu, Chunjing Xu, Qi Tian, and Chang Xu. Cars: Continuous evolution for efficient neural architecture search. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1829–1838, 2020.

[24] Xiangning Chen and Cho-Jui Hsieh. Stabilizing differentiable architecture search via perturbation-based regularization. In *International conference on machine learning*, pages 1554–1565. PMLR, 2020.

[25] Shan You, Tao Huang, Mingmin Yang, Fei Wang, Chen Qian, and Changshui Zhang. Greedynas: Towards fast one-shot nas with greedy supernet. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1999–2008, 2020.

[26] Ganesh Jawahar, Subhabrata Mukherjee, Xiaodong Liu, Young Jin Kim, Muhammad Abdul-Mageed, Laks VS Lakshmanan, Ahmed Hassan Awadallah, Sebastien Bubeck, and Jianfeng Gao. Automoe: Heterogeneous mixture-of-experts with adaptive computation for efficient neural machine translation. *arXiv preprint arXiv:2210.07535*, 2022.

[27] Shihao Han, Sishuo Liu, Shucheng Du, Mingzi Li, Zijian Ye, Xiaoxin Xu, Yi Li, Zhongrui Wang, and Dashan Shang. Cmn: a co-designed neural architecture search for efficient computing-in-memory-based mixture-of-experts. *Science China Information Sciences*, 67(10):200405, 2024.

[28] Lotfi Abdelkrim Mecharbat, Alberto Marchisio, Muhammad Shafique, Mohammad M Ghassemi, and Tuka Alhanai. Moenas: Mixture-of-expert based neural architecture search for jointly accurate, fair, and robust edge deep neural networks. *arXiv preprint arXiv:2502.07422*, 2025.

[29] Zixiang Chen, Yihe Deng, Yue Wu, Quanquan Gu, and Yuanzhi Li. Towards understanding the mixture-of-experts layer in deep learning. *Advances in neural information processing systems*, 35:23049–23062, 2022.

[30] Tsz Chai Fung and Spark C Tseung. Mixture of experts models for multilevel data: modelling framework and approximation theory. *arXiv e-prints*, pages arXiv–2209, 2022.

[31] Jiang Guo, Darsh J Shah, and Regina Barzilay. Multi-source domain adaptation with mixture of experts. *arXiv preprint arXiv:1809.02256*, 2018.

[32] Zhiwei Hu, Víctor Gutiérrez-Basulto, Zhiliang Xiang, Ru Li, and Jeff Z Pan. Multi-level mixture of experts for multimodal entity linking. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 979–990, 2025.

[33] Hongbo Li, Sen Lin, Lingjie Duan, Yingbin Liang, and Ness B Shroff. Theory on mixture-of-experts in continual learning. *arXiv preprint arXiv:2406.16437*, 2024.

[34] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35: 7103–7114, 2022.
[35] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38, 1977.
[36] C. F. Jeff Wu. On the convergence properties of the EM algorithm. *The Annals of Statistics*, 11(1):95–103, 1983.
[37] Herbert Robbins and David Siegmund. A convergence theorem for non negative almost supermartingales and some applications. In *Optimizing Methods in Statistics*, pages 233–257. Academic Press, 1971.
[38] Olivier Cappé and Éric Moulines. On-line expectation–maximization algorithm for latent data models. *Journal of the Royal Statistical Society, Series B*, 71(3):593–613, 2009.

A Proof of Theorem 1

This appendix collects the lemmas and the full proof of Theorem 1. We follow the notation of Section 4.3: $\theta = (\boldsymbol{\alpha}_{1:K}, \mathbf{r}) \in \Theta = \mathcal{A}^K \times (\Delta_K)^M$ with $\mathcal{A}$ finite, ε_t is the per-iteration surrogate error, $\mathcal{F}_v$ is the variational free energy

$$\mathcal{F}_v(q, \theta) \;=\; \sum_{m=1}^{M} \sum_{k=1}^{K} q_{mk} \, \log \frac{r_{mk} \, v(\boldsymbol{\alpha}_k, \mathbf{R}_k)}{q_{mk}},$$

and $L_v(\theta) = \max_q \mathcal{F}_v(q, \theta)$ is the log-likelihood induced by quality function v. We write $L_{\text{true}} = L_{u_{\text{true}}}$ and $L^{(t)} = L_{u^{(t)}}$.

A.1 Variational Representation of the Log-Likelihood

Proposition 1 (ELBO maximisation). *For every quality function v and every $\theta \in \Theta$, $L_v(\theta) = \max_{q \in \mathcal{Q}} \mathcal{F}_v(q, \theta)$ with the maximum attained at $q^\star_{mk}(\theta) = r_{mk} \, v(\boldsymbol{\alpha}_k, \mathbf{R}_k) \,/\, \sum_j r_{mj} \, v(\boldsymbol{\alpha}_j, \mathbf{R}_j)$.*

Proof. Fix θ and m. Writing $a_{mk} = r_{mk} \, v(\boldsymbol{\alpha}_k, \mathbf{R}_k)$ and using $\sum_k q_{mk} = 1$, the inner term equals $-\mathrm{KL}(q_{m,:} \| a_{m,:} / \sum_j a_{mj}) + \log \sum_j a_{mj}$. The KL-divergence is non-negative and vanishes exactly at $q_{mk} = a_{mk} / \sum_j a_{mj}$, which yields the claim. □

A consequence is the Neal–Hinton decomposition $L_v(\theta) = \mathcal{F}_v(q, \theta) + \sum_m \mathrm{KL}\big(q_{m,:} \| q^\star_{m,:}(\theta)\big)$, implying $L_v(\theta) \geq \mathcal{F}_v(q, \theta)$ with equality iff $q = q^\star(\theta)$.

A.2 Uniform ELBO and Likelihood Deviation

Lemma 1 (Uniform ELBO deviation). *Under Assumption 1, for every q, θ, t, $\big|\mathcal{F}_{u_{\text{true}}}(q, \theta) - \mathcal{F}_{u^{(t)}}(q, \theta)\big| \leq M \, \varepsilon_t$.*

Proof. The terms $\log r_{mk}$ and $-\log q_{mk}$ cancel between the two ELBOs, leaving $\mathcal{F}_{u_{\text{true}}} - \mathcal{F}_{u^{(t)}} = \sum_m \sum_k q_{mk} \big[\log u_{\text{true}} - \log u^{(t)}\big]$. Bounding $|\log u_{\text{true}} - \log u^{(t)}| \leq \varepsilon_t$ and using $\sum_k q_{mk} = 1$ gives the claim. □

Corollary 1. *For every θ, $|L_{\text{true}}(\theta) - L^{(t)}(\theta)| \leq M \, \varepsilon_t$.*

Proof. Apply Lemma 1 to the maxima of the two ELBOs and use $|\max_q f - \max_q g| \leq \sup_q |f - g|$. □

A.3 Quasi-Monotonicity

Proposition 2 (Quasi-monotonicity of L_{true}). *Under Assumptions 1–3, for every t,*

$$L_{\text{true}}(\theta^{(t+1)}) \;\geq\; L_{\text{true}}(\theta^{(t)}) \;-\; 2M\,\varepsilon_t. \tag{14}$$

Proof. We chain five inequalities. First, $L_{\text{true}}(\theta^{(t+1)}) \geq \mathcal{F}_{u_{\text{true}}}(q^{(t)}, \theta^{(t+1)})$ since $L_v(\theta) = \max_q \mathcal{F}_v(q, \theta)$. Second, by Lemma 1, $\mathcal{F}_{u_{\text{true}}}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t+1)}) - M\varepsilon_t$. Third, the GEM Assumption 3 yields $\mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t)})$. Fourth, the E-step makes the latter equal to $L^{(t)}(\theta^{(t)})$. Fifth, Corollary 1 gives $L^{(t)}(\theta^{(t)}) \geq L_{\text{true}}(\theta^{(t)}) - M\varepsilon_t$. Combining the five steps yields (14). □

A.4 Proof of Theorem 1(a)

Set $L_{\max} = \sup_{\theta\in\Theta} L_{\text{true}}(\theta)$, which is finite because Θ is compact (Assumption 4 and finiteness of $\mathcal{A}$). Define $V_t = L_{\max} - L_{\text{true}}(\theta^{(t)}) \geq 0$. From Proposition 2,

$$V_{t+1} \;\leq\; V_t \;+\; 2M\varepsilon_t, \qquad \sum_{t=0}^{\infty} 2M\varepsilon_t < \infty \;\text{ by Assumption 2.}$$

The Robbins–Siegmund theorem [37] (with $a_t = 0$, $b_t = 2M\varepsilon_t$) implies that $\{V_t\}$ converges, hence $\{L_{\text{true}}(\theta^{(t)})\}$ converges. □

A.5 Stability Lemmas for the M-step Analysis

To analyse limit points we need two stability statements that compare the surrogate-based and the true responsibilities.

Lemma 2 (Softmax stability). *Let $a_k, \tilde{a}_k > 0$ satisfy $e^{-\varepsilon}a_k \leq \tilde{a}_k \leq e^{\varepsilon}a_k$. Then the normalised vectors $p_k = a_k/\sum_j a_j$ and $\tilde{p}_k = \tilde{a}_k/\sum_j \tilde{a}_j$ obey $|\tilde{p}_k - p_k| \leq (e^{2\varepsilon} - 1)\,p_k \leq 3\varepsilon\,p_k$ for $\varepsilon \leq 1$.*

Proof. $\tilde{p}_k \leq a_k e^{\varepsilon}/\sum_j a_j e^{-\varepsilon} = p_k\, e^{2\varepsilon}$ and similarly $\tilde{p}_k \geq p_k\, e^{-2\varepsilon}$. □

Lemma 3 (Responsibility deviation). *Let $w_{mk}(\theta) = q^{\star}_{mk}(\theta)$ be the true-surrogate responsibilities (Proposition 1 with $v = u_{\text{true}}$) and $q^{(t)}_{mk}$ the surrogate responsibilities of the E-step at iteration t. For $\varepsilon_t \leq 1$, $|q^{(t)}_{mk} - w_{mk}(\theta^{(t)})| \leq 3\varepsilon_t\, w_{mk}(\theta^{(t)})$.*

Proof. Apply Lemma 2 with $a_{mk} = r^{(t)}_{mk}\, u_{\text{true}}(\boldsymbol{\alpha}^{(t)}_k, \mathbf{R}^{(t)}_k)$ and $\tilde{a}_{mk}$ obtained by replacing u_{true} by $u^{(t)}$. □

Lemma 4 (Q-function deviation). *Define $\tilde{Q}^{(t)}(\theta;\theta') = \sum_{m,k} q^{(t)}_{mk}(\theta') \log\big(r_{mk}\, u^{(t)}(\boldsymbol{\alpha}_k, \mathbf{R}_k)\big)$. Under Assumption 1 there exists a constant $C_Q = C_Q(M, K, c, C)$ such that for every θ, θ' and every t with $\varepsilon_t \leq 1$, $|\tilde{Q}^{(t)}(\theta;\theta') - Q_{\text{true}}(\theta;\theta')| \leq C_Q\,\varepsilon_t$.*

Proof. Add and subtract $\sum_{m,k} q^{(t)}_{mk} \log\big(r_{mk} u_{\text{true}}(\boldsymbol{\alpha}_k, \mathbf{R}_k)\big)$. The first piece, the surrogate-error term, is bounded by $M\varepsilon_t$. The remainder splits as $\sum_{m,k}(q^{(t)}_{mk} - w_{mk}) \log u_{\text{true}}$ and $\sum_{m,k}(q^{(t)}_{mk} - w_{mk}) \log r_{mk}$. The first is bounded by $3MK\Lambda\varepsilon_t$ with $\Lambda = \max(|\log c|, |\log C|)$ via Lemma 3. For the second, using $|x \log x| \leq 1/e$ on $[0, 1]$ and the $3\varepsilon_t/(cK) \cdot C$ bound on the inner softmax difference (Lemma 2), one obtains a bound of $3MC\varepsilon_t/(ec)$. Summing the three contributions gives the claim with $C_Q = M(1 + 3K\Lambda + 3C/(ec))$. □

A.6 Proof of Theorem 1(b)

Step 1: existence. Θ is compact, so $\{\theta^{(t)}\}$ has at least one limit point $\theta^{\star} = \lim_{j\to\infty} \theta^{(t_j)}$.

Step 2: contradiction setup. Suppose $\theta^\star$ is not stationary: there is $\theta' \in \Theta$ and $\delta > 0$ with $Q_{\text{true}}(\theta'; \theta^\star) - Q_{\text{true}}(\theta^\star; \theta^\star) = \delta > 0$.

Step 3: continuity. The map $\theta' \mapsto Q_{\text{true}}(\theta; \theta')$ is continuous on Θ: $w_{mk}(\theta')$ is continuous because u_{true} is continuous in $\mathbf{R}$ (Assumption 4) and softmax is continuous; and the product $w_{mk} \log r_{mk}$ extends continuously to $r_{mk} = 0$ because $w_{mk} = O(r_{mk})$ and $r \mapsto r \log r$ is continuous at 0. Thus, for j large enough, $Q_{\text{true}}(\theta'; \theta^{(t_j)}) - Q_{\text{true}}(\theta^{(t_j)}; \theta^{(t_j)}) \geq \delta/2$.

Step 4: transfer to $\tilde{Q}$. By Lemma 4,

$$\tilde{Q}^{(t_j)}(\theta'; \theta^{(t_j)}) - \tilde{Q}^{(t_j)}(\theta^{(t_j)}; \theta^{(t_j)}) \;\geq\; \delta/2 - 2C_Q\, \varepsilon_{t_j} \;\geq\; \delta/4$$

for j large enough, since $\varepsilon_{t_j} \to 0$.

Step 5: GEM ascent. Because the M-step searches over a candidate set that contains θ' (by finiteness of $\mathcal{A}$ and the closure properties of the $\mathbf{r}$-update), $\tilde{Q}^{(t_j)}(\theta^{(t_j+1)}; \theta^{(t_j)}) \geq \tilde{Q}^{(t_j)}(\theta'; \theta^{(t_j)})$. Translating back through Lemma 4 once more,

$$\Delta_{t_j} := Q_{\text{true}}(\theta^{(t_j+1)}; \theta^{(t_j)}) - Q_{\text{true}}(\theta^{(t_j)}; \theta^{(t_j)}) \;\geq\; \delta/4 - 2C_Q\, \varepsilon_{t_j} \;\geq\; \delta/8$$

for j large enough.

Step 6: summability contradiction. The Gibbs inequality $L_{\text{true}}(\theta^{(t+1)}) - L_{\text{true}}(\theta^{(t)}) \geq \Delta_t$ together with Proposition 2 implies $\Delta_t \geq -2M\varepsilon_t$. Hence the non-negative sequence $\Delta_t + 2M\varepsilon_t$ satisfies $\sum_t (\Delta_t + 2M\varepsilon_t) < \infty$ by Robbins–Siegmund applied to V_t, and consequently $\sum_t \Delta_t < \infty$. But Step 5 gives $\Delta_{t_j} \geq \delta/8$ for infinitely many j, yielding $\sum_t \Delta_t = +\infty$ — a contradiction. Therefore $\theta^\star$ is stationary. □

A.7 Convergence Rate

A by-product of the proof of Theorem 1(a) is the explicit rate

$$L_{\text{true}}(\theta^\star) - L_{\text{true}}(\theta^{(T)}) \;\leq\; 2M \sum_{t=T}^{\infty} \varepsilon_t. \tag{15}$$

If $\varepsilon_t = O(t^{-p})$ with $p > 1$, this gives $L_{\text{true}}(\theta^{(T)}) \to L_{\text{true}}(\theta^\star)$ at rate $O(T^{1-p})$, matching the rate observed for stochastic-EM analyses [38].